Renga@DBA

Goal:

To fix a performance issue by **forcing a good execution plan** using Oracle SQL Profile in a real-time environment.

☑ What is `coe_xfr_sql_profile.sql`?

Oracle provides this script to **capture a known good execution plan** and apply it as a **SQL Profile**, which can be used to consistently force the same execution plan even if the optimizer changes.

🗀 Script Location:

`$ORACLE_HOME/rdbms/admin/coe_xfr_sql_profile.sql`
(Part of the SQLT toolset — requires SQLT to be installed)

🛠 Pre-requisites:

- You must have **DBA privileges**.
- SQLT must be installed in your database.
- You should have the **SQL_ID of both the good plan and the bad plan**.

🔍 Step-by-Step Execution:

Step 1: Identify the SQL_ID of the query

Find the SQL_ID for which you want to force a better plan:

```
SELECT sql_id, sql_text FROM v$sql WHERE sql_text LIKE
'%<part_of_your_query>%';
```

step 2: Find the plan_hash_values

List all execution plans for this SQL_ID:

```
SELECT sql_id, plan_hash_value, child_number, executions, elapsed_time,
buffer_gets
```

Renga@DBA

```
FROM v$sql
WHERE sql_id = 'your_sql_id'
ORDER BY elapsed_time;
```

👉 Pick the **plan_hash_value** with the best performance (good plan) and note the **SQL_ID**.

Step 3: Run the coe_xfr_sql_profile.sql script

Connect as a user with DBA privileges (preferably SYS or SYSTEM).

```
From OS
cd $ORACLE_HOME/rdbms/admin
sqlplus / as sysdba
@coe_xfr_sql_profile.sql
```

It will prompt for:

1. SQL_ID → Enter the SQL ID (with the good plan)
2. PLAN_HASH_VALUE → Enter the good plan hash value

Example:

```
Enter SQL_ID: 9z1fyq91b5n1d
Enter PLAN_HASH_VALUE: 3468927368
```

Step 4: Review the generated script

After running, it generates a script named like:

```
coe_xfr_sql_profile_9z1fyq91b5n1d_3468927368.sql
```

💡 **Open and review it** — It contains a call to `DBMS_SQLTUNE.IMPORT_SQL_PROFILE` to create the profile.

Step 5: Execute the generated script

Run the generated `.sql` script to apply the SQL profile:

```
@coe_xfr_sql_profile_9z1fyq91b5n1d_3468927368.sql
```

Renga@DBA

📌 This will output:

```
SQL Profile "COE_9z1fyq91b5n1d_3468927368" created.
```

Step 6: Validate the SQL is using the forced plan

Check again in AWR or v$sql to ensure the SQL is now using the desired plan:

```
SELECT sql_id, plan_hash_value, executions, elapsed_time
FROM v$sql
WHERE sql_id = 'your_sql_id';
```

Or verify the profile:

```
SELECT sql_id, sql_profile FROM dba_sql_profiles WHERE sql_id =
'your_sql_id';
```

☑ Optional: Fix plan issues if not working

If the plan is still not picked:

- Gather fresh stats.
- Ensure bind peeking/values are same.
- Drop and recreate the SQL Profile if needed:

```
EXEC DBMS_SQLTUNE.DROP_SQL_PROFILE('COE_9z1fyq91b5n1d_3468927368');
```

🧠 Real-Time Use Case Summary:

Task	Action
Identify SQL	Use `v$sql` to get SQL_ID
Get good plan hash	Use `v$sql` with `plan_hash_value`
Run coe_xfr_sql_profile	Generate profile script
Execute generated script	Create SQL profile
Validate	Ensure the plan is used

Advantages:

- Non-invasive: No code changes required
- Can be removed anytime
- Helps stabilize production queries